BurnoutExys

Release 1.0.0

Stefano Bia, Younes Laihi, Alfonso Íñiguez, Mohammed Ali

May 14, 2026

CONTENTS:

CHAPTER

ONE

APPLICATION PACKAGES

1.1 data package

1.1.1 Data Management Package

This package provides a centralized container and robust parsers for loading, validating, and managing all domain-specific data sets, mapping configurations, fuzzy linguistic variables, and logic rules required by the application.

class data.**Data**

Bases: object

Container for managing application data, including input sets, rules, variables, and inference results.

Variables

- **_data** (*pd.DataFrame | None*) – Internal Pandas DataFrame holding processed inputs.
- **_mappings** (*dict[str, str] | None*) – Column mappings from the raw CSV.
- **_rules** (*tuple[Rule, ...] | None*) – Tuple of loaded fuzzy logic rules.
- **_vars** (*dict[str, Variable] | None*) – Dictionary of loaded linguistic variables mapped by name.
- **_outvar** (*Variable | None*) – The final output linguistic variable for inference.
- **_results** (*dict[str, pd.DataFrame]*) – Dictionary caching generated results DataFrames by engine ID.

property data: DataFrame

Get processed input data.

Returns
DataFrame containing processed data.

load_csv(*path: str*) → bool

Ingest data from a CSV file.

Warning

If pandas fails to parse the CSV or the file is empty, this method will catch the exception internally and reset the `_data` attribute to `None` to prevent corrupted states.

Parameters
path (*str*) – Path to the CSV file.

Returns
True if successful, False otherwise.

Return type
bool

Raises
ValueError – Caught internally if the CSV data is empty.

`load_map`(*path: str*) → bool

Load column mappings from a .map file and apply summation to the data.

Parameters
path (*str*) – Path to the mapping file.

Returns
True if successful, False otherwise.

Return type
bool

Raises
ValueError – Caught internally if the mapping file resolves to empty.

`load_rules`(*path: str*) → bool

Load fuzzy rules from a .rules file.

Parameters
path (*str*) – Path to the rules file.

Returns
True if successful, False otherwise.

Return type
bool

Raises
ValueError – Caught internally if rules payload resolves to empty.

`load_vars`(*path: str*) → bool

Load fuzzy variables from a .vars file.

Parameters
path (*str*) – Path to the vars file.

Returns
True if successful, False otherwise.

Return type
bool

Raises
ValueError – Caught internally if variable payload resolves to empty.

`property outvar: Variable | None`

Get the output variable.

Returns
Output variable, if any, or None.

`property results: dict[str, DataFrame]`

Get results data.

Returns
Dictionary containing results DataFrames indexed by id.

`property rules: tuple[Rule, ...]`

Get the loaded fuzzy rules.

Returns
Tuple of Rule objects.

store(*results: DataFrame*, *engine:* type*[object]*) → None

Store inference results in the container.

Parameters

- **results** (*pd.DataFrame*) – DataFrame containing the inference results.
- **engine** (type*[object]*) – Engine instance used to compute the results.

property variables: dict[str, *Variable*]

Get the loaded fuzzy variables.

Returns
Dict of Variable objects mapped by their names.

1.1.2 Submodules

1.1.3 data.data module

Data Container

This module provides the central `Data` container class. It acts as the in-memory repository for the entire application state, storing parsed rules, linguistic variables, mappings, and the raw and processed pandas DataFrames.

class data.data.**Data**

Bases: `object`

Container for managing application data, including input sets, rules, variables, and inference results.

Variables

- **_data** (*pd.DataFrame | None*) – Internal Pandas DataFrame holding processed inputs.
- **_mappings** (*dict[str, str] | None*) – Column mappings from the raw CSV.
- **_rules** (*tuple[Rule, ...] | None*) – Tuple of loaded fuzzy logic rules.
- **_vars** (*dict[str, Variable] | None*) – Dictionary of loaded linguistic variables mapped by name.
- **_outvar** (*Variable | None*) – The final output linguistic variable for inference.
- **_results** (*dict[str, pd.DataFrame]*) – Dictionary caching generated results DataFrames by engine ID.

property data: DataFrame

Get processed input data.

Returns
DataFrame containing processed data.

load_csv(*path: str*) → bool

Ingest data from a CSV file.

Warning

If pandas fails to parse the CSV or the file is empty, this method will catch the exception internally and reset the `_data` attribute to `None` to prevent corrupted states.

Parameters
path (*str*) – Path to the CSV file.

Returns
True if successful, False otherwise.

Return type
bool

Raises
ValueError – Caught internally if the CSV data is empty.

load_map(*path: str*) → bool

Load column mappings from a .map file and apply summation to the data.

Parameters
path (*str*) – Path to the mapping file.

Returns
True if successful, False otherwise.

Return type
bool

Raises
ValueError – Caught internally if the mapping file resolves to empty.

load_rules(*path: str*) → bool

Load fuzzy rules from a .rules file.

Parameters
path (*str*) – Path to the rules file.

Returns
True if successful, False otherwise.

Return type
bool

Raises
ValueError – Caught internally if rules payload resolves to empty.

load_vars(*path: str*) → bool

Load fuzzy variables from a .vars file.

Parameters
path (*str*) – Path to the vars file.

Returns
True if successful, False otherwise.

Return type
bool

Raises
ValueError – Caught internally if variable payload resolves to empty.

property outvar: *Variable* | None

Get the output variable.

Returns
Output variable, if any, or None.

property results: dict[str, DataFrame]

Get results data.

Returns
Dictionary containing results DataFrames indexed by id.

property rules: tuple[*Rule*, ...]

Get the loaded fuzzy rules.

Returns
Tuple of Rule objects.

store(*results: DataFrame*, *engine:* type*[object]*) → None

Store inference results in the container.

Parameters

- **results** (*pd.DataFrame*) – DataFrame containing the inference results.
- **engine** (type*[object]*) – Engine instance used to compute the results.

property variables: dict[str, *Variable*]

Get the loaded fuzzy variables.

Returns
Dict of Variable objects mapped by their names.

1.1.4 data.parsers module

Domain Parsers

This module provides concrete parser implementations for transforming raw text file payloads (.rules, .vars) into structured, operable domain objects such as LinguisticVariables and Abstract Syntax Trees for fuzzy logic rules.

data.parsers.**PRECISION: int = 1000**

Integer defining the resolution (number of discrete steps) used when generating numpy arrays for fuzzy universes.

class data.parsers.**Parser**

Bases: ABC

Abstract base class for parsers.

abstractmethod parse(*text: str*) → object

Parse text and return corresponding object.

Parameters
text – Text to parse.

Returns
Parsed object.

class data.parsers.**RuleParser**

Bases: *Parser*

Parser for fuzzy rules definitions.

Format: IF Var1 OR Var2 AND (Var3 OR NOT Var4) THEN OutputVar IS OutputSet

Variables

- **_pos** (*int*) – Current index position in the token list during parsing.
- **_tokens** (*list[str]*) – List of extracted string tokens from the rule antecedent.

Example

```
>>> parser = RuleParser()
>>> rule = parser.parse("IF exhaustion IS high AND cynicism IS high␣
↪THEN burnout IS high")
```

parse(*text: str*) → list[*Rule*]

Parse rule definitions from text.

Parameters
text – Text containing rule definitions.

Returns
List of parsed Rule objects.

property pos: int

Get current position.

Returns
Current position index.

property tokens: list[str]

Get current tokens.

Returns
List of tokens.

class data.parsers.**VariableParser**

Bases: *Parser*

Parser for linguistic variables definitions.

Format: VariableName<DomainStart, DomainEnd>: (FirstSet<func, val1, ..., valn>; ...; NSet<func, val2, ..., valn>)

parse(*text: str*) → list[*Variable*]

Parse variable definitions from text.

Parameters
text – Text containing variable definitions.

Returns
List of parsed Variable objects.

1.2 evaluation package

Evaluation package for calculating metrics.

class evaluation.**Evaluator**(*x_results: DataFrame | Series*, *y_results: DataFrame | Series*)

Bases: object

Compare engine results.

property mae: float

Calculate Mean Absolute Error.

Returns
MAE score.

property mse: float

Calculate Mean Squared Error.

Returns
MSE score.

1.2.1 Submodules

1.2.2 evaluation.metrics module

Evaluation metrics module.

class evaluation.metrics.**Evaluator**(*x_results: DataFrame | Series*, *y_results: DataFrame | Series*)

Bases: object

Compare engine results.

property mae: float

Calculate Mean Absolute Error.

Returns
MAE score.

property mse: float

Calculate Mean Squared Error.

Returns
MSE score.

1.3 inference package

1.3.1 Inference Subsystem

This package contains the core intelligence of the application. It provides the base infrastructure and specialized engines capable of executing evaluations, whether through deterministic rules (MBI) or advanced logical operators over fuzzy sets.

1.3.2 Subpackages

inference.engines package

Inference Engines

This package houses concrete implementations of inference strategies that derive burnout and risk scores from processed data. It utilizes the Strategy Pattern, a design approach that allows the system to seamlessly swap out different evaluation algorithms (like deterministic MBI logic versus fuzzy logic) at runtime without altering the internal code that leverages them.

class inference.engines.**BaseEngine**

Bases: ABC

Abstract base class for inference engines.

Variables

- **_meta** (Data) – Reference to the application's Data container for evaluating rules.
- **_results** (*pd.DataFrame*) – Cached pandas DataFrame containing the computed inference results.
- **_colnames** (*tuple[str, ...]*) – Tuple of variable names forming the columns of the results table.

evaluate(*data:* Data, *progress_fn: Callable[[int, int], None] | None = None*) → None

Evaluate input data and store results.

Parameters

- **data** (Data) – Input Data containing raw/processed data.
- **progress_fn** (*Callable[[int, int], None] | None*) – Optional callback for reporting progress (current, total).

abstract property results: DataFrame

class inference.engines.**FuzzyEngine**

Bases: *BaseEngine*

Inference engine based on fuzzy logic.

Variables

- **_input_variables** (*dict[str, Variable]*) – Dictionary of linguistic variables used as inputs.
- **_output_variable** (*Variable*) – The linguistic variable representing the output.
- **_rules** (*tuple[Rule, ...] | list[Rule]*) – Tuple or list of fuzzy rules guiding the inference process.

property **results: DataFrame**

Get the inference results.

Returns

DataFrame containing Fuzzy engine results.

class inference.engines.**MBIEngine**

Bases: *BaseEngine*

Inference engine based on Maslach Burnout Inventory deterministic rules.

Variables

ranges (*dict[str, tuple[float, float]]*) – Dictionary mapping MBI dimension groups to their range delimiters for discerning between 'Low', 'Mid' and 'High'

ranges: dict[str, tuple[float, float]] = {'AgotamientoEmocional': (11.25, 33.75), 'Despersonalizacion': (6.25, 18.75), 'RealizacionPersonal': (10, 30)}

property **results: DataFrame**

Get the inference results.

Returns

DataFrame containing MBI results.

Submodules

inference.engines.base_engine module

Base Inference Engine

This module provides the foundational architecture for all inference engines in the application. It employs the Template Method design pattern to dictate the generic, step-by-step algorithm for processing data row by row during evaluation. Child classes inherit this base and override specific hooks (initialization, single-row evaluation, and result storage) to inject their own distinct logic.

class inference.engines.base_engine.**BaseEngine**

Bases: ABC

Abstract base class for inference engines.

Variables

- **_meta** (*Data*) – Reference to the application's Data container for evaluating rules.
- **_results** (*pd.DataFrame*) – Cached pandas DataFrame containing the computed inference results.
- **_colnames** (*tuple[str, ...]*) – Tuple of variable names forming the columns of the results table.

evaluate(*data:* Data, *progress_fn: Callable[[int, int], None] | None = None*) → None

Evaluate input data and store results.

Parameters

- **data** (Data) – Input Data containing raw/processed data.
- **progress_fn** (*Callable[[int, int], None] | None*) – Optional callback for reporting progress (current, total).

abstract property results: DataFrame

inference.engines.fuzzy_engine module

Fuzzy Logic Engine

This module implements the core FuzzyEngine, inheriting the BaseEngine Template Method. It evaluates complex fuzzy logic rule sets against input data, computing aggregate membership activation strengths and returning defuzzified centroid scores.

class inference.engines.fuzzy_engine.**FuzzyEngine**

Bases: *BaseEngine*

Inference engine based on fuzzy logic.

Variables

- **_input_variables** (*dict[str, Variable]*) – Dictionary of linguistic variables used as inputs.
- **_output_variable** (Variable) – The linguistic variable representing the output.
- **_rules** (*tuple[Rule, ...] | list[Rule]*) – Tuple or list of fuzzy rules guiding the inference process.

property results: DataFrame

Get the inference results.

Returns

DataFrame containing Fuzzy engine results.

inference.engines.mbi_engine module

Deterministic MBI Engine

This module implements the MBIEngine, inheriting the BaseEngine Template Method. It evaluates raw data using standard deterministic Maslach Burnout Inventory thresholds to output categorical risk classifications ('Low', 'Mid', 'High').

class inference.engines.mbi_engine.**MBIEngine**

Bases: *BaseEngine*

Inference engine based on Maslach Burnout Inventory deterministic rules.

Variables

ranges (*dict[str, tuple[float, float]]*) – Dictionary mapping MBI dimension groups to their range delimiters for discerning between 'Low', 'Mid' and 'High'

ranges: dict[str, tuple[float, float]] = {'AgotamientoEmocional': (11.25, 33.75), 'Despersonalizacion': (6.25, 18.75), 'RealizacionPersonal': (10, 30)}

property results: DataFrame

Get the inference results.

Returns

DataFrame containing MBI results.

inference.fuzzy package

Fuzzy Logic Subsystem

This package constructs a domain-specific language (DSL) and Abstract Syntax Tree (AST) for defining and evaluating fuzzy logic constraints, operators, rules, and linguistic variables across mathematical universes.

type inference.fuzzy.**BinaryOp = Callable[[ndarray, ndarray], ndarray]**

class inference.fuzzy.**BranchNode**(*left:* Node, *right:* Node | *None* = *None*)

Bases: *Node*, ABC

Branch node representing a logical operation between nodes.

Variables

- **_left** (Node) – Left child node.
- **_right** (Node | *None*) – Optional right child node.

class inference.fuzzy.**LeafNode**(*variable: str*, *term: str*)

Bases: *Node*

A leaf node representing a single fuzzy variable evaluation.

Variables

- **_variable** (*str*) – Name of the variable in the context.
- **_term** (*str*) – Name of the membership function to evaluate.

evaluate(*context: dict[str, dict[str, ndarray]]*) → ndarray

Evaluate the leaf node by fetching membership from context.

Parameters
context (*dict[str, dict[str, np.ndarray]]*) – Dictionary containing evaluated memberships.

Returns
Membership value.

Return type
np.ndarray

Raises
KeyError – If the variable or term is not found in the context.

class inference.fuzzy.**Node**

Bases: ABC

Abstract base class for all nodes in the evaluation tree.

abstractmethod evaluate(*context: dict[str, dict[str, ndarray]]*) → ndarray

Evaluate the node given a context.

Parameters
context (*dict[str, dict[str, np.ndarray]]*) – Dictionary containing fuzzified data for evaluation.

Returns
Evaluation results.

Return type
np.ndarray

class inference.fuzzy.**Rule**(*antecedent:* Node, *consequent:* LeafNode)

Bases: object

Represent a fuzzy IF-THEN rule.

property antecedent: *Node*

Get the antecedent term.

Returns
Antecedent term Node.

property consequent: *LeafNode*

Get the consequent term.

Returns
Consequent term Node.

evaluate(*context: dict[str, dict[str, ndarray]]*) → ndarray

Evaluate rule.

Parameters
context – Fuzzified input data.

Returns
Activation strength of the rule.

type inference.fuzzy.**UnaryOp = Callable[[ndarray], ndarray]**

class inference.fuzzy.**Variable**(*name: str*, *universe: ndarray*)

Bases: object

Represent a linguistic fuzzy variable with membership functions.

add_term(*term: str*, *membership_fn: ndarray*) → None

Add a linguistic term and its membership function.

Parameters

- **term** – Name of the term (e.g., 'low', 'high').
- **membership_fn** – Array of membership values corresponding to the universe.

fuzzify(*values: float | ndarray*) → dict[str, ndarray]

Calculate membership degrees for input value.

Parameters
value – Input value or values to fuzzify.

Returns
Dictionary of term names to membership arrays.

property name: str

Get variable name.

property terms: dict[str, ndarray]

Get variable terms.

property universe: ndarray

Get variable universe.

Submodules

inference.fuzzy.ast module

AST for fuzzy rule evaluation.

class inference.fuzzy.ast.**AndNode**(*left:* Node, *right:* Node | *None* = *None*)

Bases: *BranchNode*

Represent a fuzzy AND operation.

evaluate(*context: dict[str, dict[str, ndarray]]*) → ndarray

Evaluate the fuzzy AND using np.fmin.

Parameters
context (*dict[str, dict[str, np.ndarray]]*) – Dictionary containing data/context for evaluation.

Returns
Minimum membership value.

Return type
np.ndarray

Raises
AssertionError – If the right child node is missing.

type inference.fuzzy.ast.**BinaryOp = Callable[[ndarray, ndarray], ndarray]**

Type alias for a binary operation over numpy arrays.

class inference.fuzzy.ast.**BranchNode**(*left:* Node, *right:* Node | *None = None*)

Bases: *Node*, ABC

Branch node representing a logical operation between nodes.

Variables

- **_left** (Node) – Left child node.
- **_right** (Node | *None*) – Optional right child node.

class inference.fuzzy.ast.**LeafNode**(*variable: str*, *term: str*)

Bases: *Node*

A leaf node representing a single fuzzy variable evaluation.

Variables

- **_variable** (*str*) – Name of the variable in the context.
- **_term** (*str*) – Name of the membership function to evaluate.

evaluate(*context: dict[str, dict[str, ndarray]]*) → ndarray

Evaluate the leaf node by fetching membership from context.

Parameters
context (*dict[str, dict[str, np.ndarray]]*) – Dictionary containing evaluated memberships.

Returns
Membership value.

Return type
np.ndarray

Raises
KeyError – If the variable or term is not found in the context.

class inference.fuzzy.ast.**Node**

Bases: ABC

Abstract base class for all nodes in the evaluation tree.

abstractmethod evaluate(*context: dict[str, dict[str, ndarray]]*) → ndarray

Evaluate the node given a context.

Parameters
context (*dict[str, dict[str, np.ndarray]]*) – Dictionary containing fuzzified data for evaluation.

Returns
Evaluation results.

Return type
np.ndarray

class inference.fuzzy.ast.**NotNode**(*child:* Node)

Bases: *BranchNode*

Represent a fuzzy NOT operation.

evaluate(*context: dict[str, dict[str, ndarray]]*) → ndarray

Evaluate fuzzy NOT using 1.0 - val.

Parameters
context (*dict[str, dict[str, np.ndarray]]*) – Dictionary containing data/context for evaluation.

Returns
Inverted membership value.

Return type
np.ndarray

inference.fuzzy.ast.**OPERATORS: dict[str, *UnaryOp* | *BinaryOp*] = {'AND': <ufunc 'fmin'>, 'NOT': <function <lambda>>, 'OR': <ufunc 'fmax'>}**

Dictionary mapping string identifiers to their corresponding logical numpy operations.

class inference.fuzzy.ast.**OrNode**(*left:* Node, *right:* Node | *None* = *None*)

Bases: *BranchNode*

Represent a fuzzy OR operation.

evaluate(*context: dict[str, dict[str, ndarray]]*) → ndarray

Evaluate the fuzzy OR using np.fmax.

Parameters
context (*dict[str, dict[str, np.ndarray]]*) – Dictionary containing data/context for evaluation.

Returns
Maximum membership value.

Return type
np.ndarray

Raises
AssertionError – If the right child node is missing.

type inference.fuzzy.ast.**UnaryOp = Callable[[ndarray], ndarray]**

Type alias for a unary operation over numpy arrays.

inference.fuzzy.rules module

Fuzzy Logic Rules

This module defines the `Rule` and `ThenNode` objects, which tie together an antecedent (an AST of conditions) and a consequent (the resulting output term) to compute final activation strengths using numpy array operations.

inference.fuzzy.rules.**OPERATORS: dict[str, *BinaryOp*] = {'THEN': <ufunc 'fmin'>}**

Dictionary mapping consequent string operators to their underlying numpy aggregation functions.

class inference.fuzzy.rules.**Rule**(*antecedent:* Node, *consequent:* LeafNode)

Bases: `object`

Represent a fuzzy IF-THEN rule.

property antecedent: *Node*

Get the antecedent term.

Returns
Antecedent term Node.

property consequent: *LeafNode*

Get the consequent term.

Returns
Consequent term Node.

evaluate(*context: dict[str, dict[str, ndarray]]*) → ndarray

Evaluate rule.

Parameters
context – Fuzzified input data.

Returns
Activation strength of the rule.

class inference.fuzzy.rules.**ThenNode**(*left:* Node, *right:* Node | *None = None*)

Bases: *BranchNode*

Represent a fuzzy THEN operation.

evaluate(*context: dict[str, dict[str, ndarray]]*) → ndarray

Evaluate the fuzzy THEN operation using np.fmin.

Parameters
context – Dictionary containing data/context for evaluation.

Returns
Activation strength as a membership function.

inference.fuzzy.variables module

Fuzzy Linguistic Variables

This module encapsulates the representation of fuzzy linguistic variables, including their mathematically defined universe of discourse and associated membership functions (e.g., 'low', 'high') used during fuzzification.

class inference.fuzzy.variables.**Variable**(*name: str*, *universe: ndarray*)

Bases: object

Represent a linguistic fuzzy variable with membership functions.

add_term(*term: str*, *membership_fn: ndarray*) → None

Add a linguistic term and its membership function.

Parameters

- **term** – Name of the term (e.g., 'low', 'high').
- **membership_fn** – Array of membership values corresponding to the universe.

fuzzify(*values: float | ndarray*) → dict[str, ndarray]

Calculate membership degrees for input value.

Parameters
value – Input value or values to fuzzify.

Returns
Dictionary of term names to membership arrays.

property name: str

Get variable name.

property terms: dict[str, ndarray]

Get variable terms.

property universe: ndarray

Get variable universe.

1.4 main module

1.4.1 Application Entry Point

This module serves as the primary entry point for the Fuzzy Expert System. It encapsulates the bootstrap logic required to instantiate the interactive terminal Menu and delegates execution control to the UI package.

main.**main**() → None

Bootstrap and start the interactive terminal application.

This function instantiates the root Menu component and hands over the main execution thread to its event loop, returning the final status code upon exit.

1.5 ui package

1.5.1 User Interface Package

This package encapsulates the interactive terminal views, the main application controller, and utilities for rich console rendering and interactive file browsing.

class ui.**Menu**

Bases: object

Interactive menu for the application.

Variables

- **options** (*dict[str, str]*) – Dictionary mapping display names of menu options to their corresponding Controller methods.
- **status_items** (*dict[str, tuple[str, str]]*) – Dictionary mapping state keys to a tuple of display title and default value for the status bar.
- **_console** (*Console*) – Active Rich console instance.
- **_status** (Display) – Dedicated Display instance for rendering the static status bar.
- **_rules** (Display) – Dedicated Display instance for rendering loaded rules summary.
- **_vars** (Display) – Dedicated Display instance for rendering loaded variables summary.
- **_output** (Display) – Dedicated Display instance for ad-hoc command responses and warnings.
- **_controller** (Controller) – Central application Controller managing state logic.

cls() → None

Clear the console screen.

property data: *Data*

Get the underlying data container.

Returns

Data container instance.

echo(*string: str | ConsoleRenderable = ''*) → None

Print to the console.

Parameters
string – String to print.

options: dict[str, str] = {'Execute Inference': 'execute_inference', 'Generate Visualizations': 'generate_visualizations', 'Load Data': 'load_data', 'Load Mappings': 'load_mappings', 'Load Rules': 'load_rules', 'Load Variables': 'load_variables', 'Switch Engine': 'switch_engine'}

read(*prompt: str = ''*) → str

Read input from the console.

Parameters
prompt – Input prompt.

Returns
User input.

run() → int

Execute interactive main menu loop.

Returns
Exit status code.

property state: dict[str, str | *type*[*BaseEngine*] | None]

Get the current application state.

Returns
State dictionary.

status_items: dict[str, tuple[str, str]] = {'data': ('Data', 'No'), 'engine': ('Engine', 'None'), 'mappings': ('Mappings', 'No'), 'rules': ('Rules', 'No'), 'variables': ('Variables', 'No')}

1.5.2 Submodules

1.5.3 ui.browser module

File Browser Component

This module provides the `FileBrowser` class, which utilizes the *questionary* library to render an interactive, terminal-based file navigation interface. It allows users to traverse the local filesystem and select specific files filtered by extension.

class ui.browser.**FileBrowser**(*path: str*, *extensions: tuple[str, ...]*)

Bases: `object`

Provide interactive file browser using questionary.

Variables

- **icons** (*dict[str, str]*) – Dictionary mapping item types to their corresponding terminal icon strings.
- **_extensions** (*tuple[str, ...]*) – Tuple of allowed file extensions.
- **_items** (*list[str] | None*) – Cached list of all directory items.
- **_folders** (*list[str] | None*) – Cached list of folder names.
- **_files** (*list[str] | None*) – Cached list of file names filtered by allowed extensions.
- **_choices** (*list[str] | None*) – Cached list of questionary Choice strings.

- **_cd** (*str*) – Absolute path to the currently active directory.

property cd: str

Get current directory path.

Returns
Current directory path string.

property choices: list[str]

Get menu choices for questionary.

Returns
List of choice strings.

property extensions: tuple[str, ...]

Get allowed file extensions.

Returns
Tuple of extension strings.

property files: list[str]

Get filtered files in current directory.

Returns
List of file names.

property folders: list[str]

Get folders in current directory.

Returns
List of folder names.

icons: dict[str, str] = {'exit': '»', 'file': '[FILE]', 'folder': '[DIR]'}

into(*directory: str*) → None

Navigate into child directory.

Parameters
directory – Target directory name.

property items: list[str]

Get all items in current directory.

Returns
List of item names.

run() → str | None

Execute interactive file browser loop.

Note

This method takes control of the terminal until the user makes a valid selection or chooses to cancel the prompt.

Returns
Selected file path or None if cancelled.

Return type
str | None

up() → None

Navigate to parent directory.

1.5.4 ui.chooser module

Interactive checkbox chooser module.

class ui.chooser.**Chooser**(*min_n: int*, *max_n: int*, *choices: tuple[dict[str, Any], ...]*, *console: Console*)

Bases: object

Interactive checkbox chooser for selecting items from a list.

run() → list[str] | None

Execute the interactive chooser.

Returns

List of selected values, or None if no valid selection was made.

1.5.5 ui.controller module

Application Controller

This module defines the central `Controller` responsible for orchestrating the application's primary workflow. It acts as the mediator between the interactive terminal views (Menu, Browser, Display) and the backend logic (Data, Inference Engines), ensuring state consistency and managing the execution lifecycle.

class ui.controller.**Controller**(*console: Console*, *display:* Display)

Bases: object

Controller orchestrating the application flow.

Variables

- **engines** (*tuple[type[BaseEngine], ...]*) – Tuple of available BaseEngine subclasses that can be dynamically swapped.
- **_data** (Data) – The central Data container tracking all parsed values.
- **_display** (Display) – System-wide display controller for rendering outputs.
- **_progressbar** (ProgressBar) – System-wide progress bar utility for long-running inferences.
- **_engine** (*int*) – Index representing the currently active evaluation engine.
- **_browsers** (*dict[str, FileBrowser]*) – Dictionary mapping domain file types to their FileBrowser prompts.
- **_state** (*dict[str, str | type[BaseEngine] | None]*) – Application state dictionary accessible across the UI layer.

property data: *Data*

Get the data container.

Returns

Data container instance.

engines: tuple[*type*[*BaseEngine*], ...] = (<class 'inference.engines.mbi_engine.MBIEngine'>, <class 'inference.engines.fuzzy_engine.FuzzyEngine'>)

execute_inference() → None

Execute inference on loaded data.

generate_visualizations() → None

Generate and save configured visualizations.

load_data() → bool

Load data into the container.

Returns
True if successful, False otherwise.

load_mappings() → bool

Load mapping into the container.

Returns
True if successful, False otherwise.

load_rules() → bool

Load fuzzy rules from a .rules file.

Returns
True if successful, False otherwise.

load_variables() → bool

Load fuzzy variables from a .vars file.

Returns
True if successful, False otherwise.

property state: dict[str, str | *type*[*BaseEngine*] | None]

Get current application state.

Returns
State dictionary.

switch_engine() → None

Switch between available inference engines.

1.5.6 ui.display module

Console display utility module.

class ui.display.**Display**(*title: str*, *console: Console*)

Bases: object

Console display manager for rendering messages, warnings, and panels.

Variables

- **styles** (*dict[str, dict[str, str]]*) – Global dictionary mapping semantic types to rich styling parameters.
- **_console** (*Console*) – Active Rich console for rendering text.
- **_title** (*str | None*) – Optional title string for rendered panels.
- **_type** (*str*) – Semantic type defining the current rendering style (e.g., 'error', 'success').
- **_message** (*str | None*) – Content body to be displayed upon invocation.

echo(*string: str | ConsoleRenderable = ''*) → None

Print a raw string to the console.

Parameters
string – String to print.

set(*message: str*, *type: str | None = None*) → None

Set a formatted message to be displayed later.

Only the last set message will be displayed and messages will be displayed only once.

Parameters

- **message** – Content of the message.
- **type** – Optional type for styling the message.

show() → None

Render the configured message, if any, inside a styled panel, then reset state.

property style: dict[str, str]

Get the current style configuration.

Returns

Dictionary containing color specifications.

styles: dict[str, dict[str, str]] = {'error': {'border': 'bold red', 'text': 'red', 'title': 'bold red'}, 'normal': {'border': 'bold blue', 'text': 'blue', 'title': 'bold blue'}, 'success': {'border': 'bold green', 'text': 'green', 'title': 'bold green'}, 'warning': {'border': 'bold orange', 'text': 'orange', 'title': 'bold orange'}}

property type: str

Get the active message type (e.g., error, normal).

Returns

String representing the message type.

1.5.7 ui.menu module

Interactive terminal menu.

class ui.menu.**Menu**

Bases: object

Interactive menu for the application.

Variables

- **options** (*dict[str, str]*) – Dictionary mapping display names of menu options to their corresponding Controller methods.
- **status_items** (*dict[str, tuple[str, str]]*) – Dictionary mapping state keys to a tuple of display title and default value for the status bar.
- **_console** (*Console*) – Active Rich console instance.
- **_status** (Display) – Dedicated Display instance for rendering the static status bar.
- **_rules** (Display) – Dedicated Display instance for rendering loaded rules summary.
- **_vars** (Display) – Dedicated Display instance for rendering loaded variables summary.
- **_output** (Display) – Dedicated Display instance for ad-hoc command responses and warnings.
- **_controller** (Controller) – Central application Controller managing state logic.

cls() → None

Clear the console screen.

property data: *Data*

Get the underlying data container.

Returns

Data container instance.

echo(*string: str | ConsoleRenderable = ''*) → None

Print to the console.

Parameters
string – String to print.

options: dict[str, str] = {'Execute Inference': 'execute_inference', 'Generate Visualizations': 'generate_visualizations', 'Load Data': 'load_data', 'Load Mappings': 'load_mappings', 'Load Rules': 'load_rules', 'Load Variables': 'load_variables', 'Switch Engine': 'switch_engine'}

read(*prompt: str = ''*) → str

Read input from the console.

Parameters
prompt – Input prompt.

Returns
User input.

run() → int

Execute interactive main menu loop.

Returns
Exit status code.

property state: dict[str, str | *type*[*BaseEngine*] | None]

Get the current application state.

Returns
State dictionary.

status_items: dict[str, tuple[str, str]] = {'data': ('Data', 'No'), 'engine': ('Engine', 'None'), 'mappings': ('Mappings', 'No'), 'rules': ('Rules', 'No'), 'variables': ('Variables', 'No')}

1.5.8 ui.progress module

Progress Bar Utility

This module provides a context manager for rendering an interactive progress bar in the terminal. It leverages the rich library to display real-time progress updates, spinners, and task completion metrics during long-running operations.

class ui.progress.**ProgressBar**

Bases: object

Context manager for a Rich progress bar.

1.6 visualization package

1.6.1 Visualization Package

This package provides utilities for generating and rendering high-quality 2D plots of fuzzy membership functions, aggregated evaluation areas, and model comparison metrics using matplotlib and seaborn.

class visualization.**Plotter**(*data:* Data)

Bases: object

Utility class for generating visualizations from application Data.

Variables
_data (Data) – Central Data container tracking all parsed values and results.

plot_comparison(*key1: str*, *key2: str*, *path: str*) → None

Plot a comparison scatter plot between two engine evaluation results, overlaying the MAE and MSE metrics.

Parameters

- **key1** (*str*) – First result key.
- **key2** (*str*) – Second result key.
- **path** (*str*) – Base directory where the plot will be saved.

plot_result(*key: str*, *path: str*, *row_idx: int = 0*) → None

Plot the output variable's membership functions alongside the aggregated activation area and the final defuzzified decision.

Parameters

- **key** (*str*) – Result key representing the engine execution.
- **path** (*str*) – Base directory where the plot will be saved.
- **row_idx** (*int*) – Index of the data row to visualize.

plot_results(*key: str*, *path: str*, *progress_fn: Callable[[int, int], None] | None = None*) → None

Plot all results for a results DataFrame.

Parameters

- **key** (*str*) – Result key representing the engine execution.
- **path** (*str*) – Base directory where plots will be saved.
- **progress_fn** (*Callable[[int, int], None] | None*) – Optional callback for reporting progress (current, total).

plot_variable(*var: str*, *path: str*) → None

Plot the membership functions of a single linguistic variable.

Parameters

- **var** (*str*) – Name of the variable to plot.
- **path** (*str*) – Base directory where the plot will be saved.

plot_variables(*path: str*, *progress_fn: Callable[[int, int], None] | None = None*) → None

Plot all linguistic variables stored in the Data container.

Parameters

- **path** (*str*) – Base directory where plots will be saved.
- **progress_fn** (*Callable[[int, int], None] | None*) – Optional callback for reporting progress (current, total).

1.6.2 Submodules

1.6.3 visualization.plotters module

Visualization Plotters

This module provides utilities for rendering high-quality plots of fuzzy logic variables, rule aggregations, and inference comparisons. It relies on matplotlib and seaborn to produce static image assets.

class visualization.plotters.**Plotter**(*data: Data*)

Bases: object

Utility class for generating visualizations from application Data.

Variables
_data (Data) – Central Data container tracking all parsed values and results.

plot_comparison(*key1: str*, *key2: str*, *path: str*) → None

Plot a comparison scatter plot between two engine evaluation results, overlaying the MAE and MSE metrics.

Parameters

- **key1** (*str*) – First result key.
- **key2** (*str*) – Second result key.
- **path** (*str*) – Base directory where the plot will be saved.

plot_result(*key: str*, *path: str*, *row_idx: int = 0*) → None

Plot the output variable's membership functions alongside the aggregated activation area and the final defuzzified decision.

Parameters

- **key** (*str*) – Result key representing the engine execution.
- **path** (*str*) – Base directory where the plot will be saved.
- **row_idx** (*int*) – Index of the data row to visualize.

plot_results(*key: str*, *path: str*, *progress_fn: Callable[[int, int], None] | None = None*) → None

Plot all results for a results DataFrame.

Parameters

- **key** (*str*) – Result key representing the engine execution.
- **path** (*str*) – Base directory where plots will be saved.
- **progress_fn** (*Callable[[int, int], None] | None*) – Optional callback for reporting progress (current, total).

plot_variable(*var: str*, *path: str*) → None

Plot the membership functions of a single linguistic variable.

Parameters

- **var** (*str*) – Name of the variable to plot.
- **path** (*str*) – Base directory where the plot will be saved.

plot_variables(*path: str*, *progress_fn: Callable[[int, int], None] | None = None*) → None

Plot all linguistic variables stored in the Data container.

Parameters

- **path** (*str*) – Base directory where plots will be saved.
- **progress_fn** (*Callable[[int, int], None] | None*) – Optional callback for reporting progress (current, total).

CHAPTER

TWO

APPLICATION TEST SUITES

2.1 test_ast module

Tests for the fuzzy AST and operators.

test_ast.**test_ast_complex_structure**() → None

Test AST oeprations in a complex tree structure.

test_ast.**test_ast_evaluation_array**() → None

Test AST operations with vector values.

test_ast.**test_ast_evaluation_scalar**() → None

Test AST operations with scalar values.

2.2 test_controller module

Tests for the UI controller module.

class test_controller.**DummyEngine**

Bases: *BaseEngine*

Dummy engine for testing.

property results: DataFrame

test_controller.**test_controller_execute_inference**(*tmp_path: Path*, *mocker: MockerFixture*) → None

Test inference execution delegation.

Simulate data loading, switch to a dummy engine, and invoke inference execution. Ensure that the Controller correctly captures and stores the evaluation results.

test_controller.**test_controller_generate_visualizations**(*mocker: MockerFixture*) → None

Test visualization pipeline execution.

Verify that the Controller's generate_visualizations method executes cleanly without crashing, even when data is empty or results are absent.

test_controller.**test_controller_initialization**() → None

Test Controller instantiation.

Verify that the Controller correctly sets up its internal state dictionary, data container, and engine bindings upon initialization with a Display.

test_controller.**test_controller_load_data**(*tmp_path: Path*, *mocker: MockerFixture*) → None

Test CSV data loading through the controller.

Mock the interactive FileBrowser to simulate a user selecting a CSV file, and verify that the Controller successfully delegates the ingestion to the Data container.

test_controller.**test_controller_load_mappings**(*tmp_path: Path*, *mocker: MockerFixture*) → None

Test .map data loading through the controller.

Mock the interactive FileBrowser to simulate a user selecting a .map file, and verify that the Controller successfully delegates the ingestion to the Data container.

test_controller.**test_controller_load_rules**(*tmp_path: Path*, *mocker: MockerFixture*) → None

Test .rules data loading through the controller.

Mock the interactive FileBrowser to simulate a user selecting a .rules file, and verify that the Controller successfully delegates the ingestion to the Data container.

test_controller.**test_controller_load_variables**(*tmp_path: Path*, *mocker: MockerFixture*) → None

Test .vars data loading through the controller.

Mock the interactive FileBrowser to simulate a user selecting a .vars file, and verify that the Controller successfully delegates the ingestion to the Data container.

test_controller.**test_controller_switch_engine**() → None

Test engine switching logic in the controller.

Iterate through multiple engine switches and verify that the controller cycles through the available engine types defined in its class attribute.

2.3 test_data module

Tests for the data module.

class test_data.**DummyEngine**

Bases: *BaseEngine*

Dummy engine for testing purposes.

property results: DataFrame

test_data.**test_ingest_from_csv**(*tmp_path: Path*) → None

Test successful CSV file ingestion.

Verify that the Data container correctly reads a CSV file and stores its content in the internal DataFrame state.

test_data.**test_ingest_from_csv_failure**() → None

Test graceful failure during CSV ingestion.

Ensure that attempting to load a non-existent CSV file returns a failure status and leaves the data container empty.

test_data.**test_map**(*tmp_path: Path*) → None

Test mapping file loading and column renaming.

Verify that a .map file correctly triggers the renaming of columns in the ingested DataFrame within the Data container.

test_data.**test_map_failure**() → None

Test graceful failure during mapping file loading.

Ensure that a missing .map file is handled correctly by returning a failure status.

test_data.**test_rules**(*tmp_path: Path*) → None

Test successful loading of fuzzy rules from a file.

Verify that the RuleParser is correctly invoked to populate the internal rules tuple from a provided payload.

test_data.**test_rules_failure**() → None

Test graceful failure during rule file loading.

Ensure that missing rule files result in an empty rules tuple and a failure return status.

test_data.**test_save_results**() → None

Test inference results storage.

Verify that evaluated results from an inference engine are correctly cached within the Data container's results dictionary.

test_data.**test_vars**(*tmp_path: Path*) → None

Test successful loading of fuzzy variables from a file.

Verify that the VariableParser correctly populates the internal variables dictionary from a provided payload.

test_data.**test_vars_failure**() → None

Test graceful failure during variable file loading.

Ensure that missing variable files result in an empty variables dictionary and a failure return status.

2.4 test_engines module

Tests for inference engines.

test_engines.**test_fuzzy_engine**() → None

Test complete Fuzzy logic evaluation lifecycle.

Inject synthetic fuzzy sets (Variables), membership functions, and logical Rules into the engine. Verify that the FuzzyEngine reliably translates inputs to degrees of truth, triggers correct consequents, defuzzifies the output via centroid method, and appends the RESULTS column appropriately.

test_engines.**test_mbi_engine**() → None

Test Maslach Burnout Inventory deterministic evaluation.

Feed raw exhaustion, despersonalization and fulfillment data into the MBIEngine, and assert that it correctly classifies the risk level ('Low', 'Mid', 'High') based on the configured dimension ranges.

2.5 test_evaluation module

Tests for evaluation module.

test_evaluation.**test_calculate_discrepancy**() → None

Test MAE and MSE calculations.

Verify that the Evaluator correctly computes the Mean Absolute Error (MAE) and Mean Squared Error (MSE) between two sets of inference results.

test_evaluation.**test_calculate_discrepancy_empty**() → None

Test error metric calculation with missing data.

Ensure that the Evaluator handles empty input series gracefully by returning zeroed metrics instead of crashing.

2.6 test_parsers module

Tests for the parser module.

test_parsers.**test_rule_parser**() → None

Test Rule DSL syntax resolution.

Provide complex logical conditions and verify the parser generates the correct abstract syntax tree consisting of OrNodes, AndNodes, NotNodes, and LeafNodes bound to the correct Rule consequent object.

test_parsers.**test_rule_parser_errors**() → None

Test graceful degradation of Rule parsing.

Feed syntactically invalid strings into the Rule parser and ensure it swallows the errors gracefully, skipping malformed rules without crashing the system.

test_parsers.**test_variable_parser**() → None

Test Variable DSL syntax resolution.

Supply a raw multiline string of custom DSL Variable syntax and verify that the parser constructs valid Variables with correctly scoped numpy universes and correctly bound scikit-fuzzy membership functions.

test_parsers.**test_variable_parser_errors**() → None

Test graceful degradation of Variable parsing.

Feed syntactically invalid strings into the Variable parser and ensure it swallows the errors gracefully, skipping malformed variables without crashing the system.

2.7 test_plotters module

Tests for the visualization module.

test_plotters.**test_plot_comparison**(*tmp_path: Path*) → None

Test comparative plotting of two engine results.

Verify Plotter correctly generates a scatter plot contrasting two sets of inference results along with MAE and MSE metrics.

test_plotters.**test_plot_result**(*tmp_path: Path*) → None

Test plotting of an inference result row.

Verify Plotter correctly overlays an aggregated activation area and defuzzified decision line onto output variable's terms.

test_plotters.**test_plot_results**(*tmp_path: Path*, *mocker: MockerFixture*) → None

Test batch plotting of all rows in a result set.

Verify Plotter correctly iterates through all rows in a specified result DataFrame and delegates plotting to plot_result.

test_plotters.**test_plot_variable**(*tmp_path: Path*) → None

Test plotting of a single linguistic variable.

Verify that the Plotter correctly generates and saves a membership function plot for a specified variable.

test_plotters.**test_plot_variables**(*tmp_path: Path*, *mocker: MockerFixture*) → None

Test batch plotting of all variables.

Verify that calling plot_variables iterates through all stored variables and delegates to plot_variable.

CHAPTER

THREE

INDICES AND SEARCH

- genindex
- modindex
- search

PYTHON MODULE INDEX

INDEX

I

L

M

N

O

P

R

S

T

U

V